

Roteiro Prático de Git e GitHub: Fluxo de Ramificação e Integração

Este guia prático foi desenvolvido para estruturar e orientar os estudantes no aprendizado dos comandos e fluxos de trabalho fundamentais de controle de versão utilizando o Git e a plataforma GitHub. O objetivo primordial é simular de forma realista a rotina de uma equipe de engenharia e desenvolvimento de software, exercitando de ponta a ponta a criação de ramificações isoladas (branches), o monitoramento de novos arquivos, o registro histórico de modificações (commits), a publicação na nuvem (push) e a unificação final do código (merge).

1. Objetivos de Aprendizagem

Ao concluir as etapas propostas neste laboratório, o estudante deverá ser capaz de:

- Compreender a necessidade do uso de ramificações para a preservação da estabilidade da linha principal de produção (branch main).
- Manipular e interpretar os diferentes estados de um arquivo dentro do ciclo de vida local do Git (Untracked, Staged e Committed).
- Estabelecer a conexão correta entre o ambiente de desenvolvimento local e os repositórios hospedados no GitHub.
- Executar o fluxo completo de resolução de demandas, isolamento de escopo e integração de código sem perda de histórico.

2. Tabela de Comandos Referenciais

A tabela a seguir apresenta os principais comandos que serão aplicados durante a atividade, servindo como uma base de consulta rápida para a sintaxe correta no terminal.

Comando Git	Ambiente	Descrição Técnica da Operação
git status	Local	Exibe o estado atual dos arquivos que sofreram modificações, adições ou que não estão sendo

Comando Git	Ambiente	Descrição Técnica da Operação
		monitorados.
git checkout -b <nome-da-branch>	Local	Cria uma nova ramificação de código com o nome especificado e altera o ambiente ativo para ela de forma imediata.
git add <caminho-do-arquivo>	Local	Transfere as alterações de um arquivo modificado da Área de Trabalho para a Área de Preparação (Staging Area).
git commit -m "<mensagem>"	Local	Grava permanentemente o pacote de alterações que estava na Área de Preparação, gerando um identificador único no histórico local.
git push origin <nome-da-branch>	Remoto	Envia a ramificação criada localmente e seu respectivo histórico de commits para o servidor remoto configurado no

Comando Git	Ambiente	Descrição Técnica da Operação
		GitHub.
git checkout <nome-da-branch>	Local	Alterna o ambiente de desenvolvimento entre ramificações já existentes no repositório local.
git merge <nome-da-branch>	Local	Une o histórico de alterações e commits contidos na ramificação especificada de encontro à branch ativa no momento.
git branch -d <nome-da-branch>	Local	Exclui uma branch local da lista de branches disponíveis

3. Passo a Passo do Exercício Prático

Siga minuciosamente as etapas sequenciais descritas abaixo para concluir o fluxo de versionamento estabelecido para cada um dos exercícios da lista 04.

Passo 1: Crie um repositório em sua conta do GitHub e abra a

Logue em sua conta e acesse a aba repositórios. Clique na opção New. A seguir execute as ações abaixo:

1. Informe o nome do repositório.
2. Escolha a visibilidade: Selecione público
3. Clique em criar repositório.

4. Clone o repositório no computador, através do comando:
 - a. `git clone <link do repositório>`
5. Abra a pasta criada no VSCode

Passo 2: Isolamento de Escopo e Criação da Branch

Utilizando o terminal integrado, crie para cada exercício da lista de exercícios 04 um branch.

```
git checkout -b Ex1.py
```

Execute o comando de verificação para garantir que seu ambiente saiu da main e agora aponta para a nova ramificação de recursos:

```
git branch
```

Passo 3: Adição de Arquivos e Mudança de Estado

Após resolvido o exercício, verifique o status do arquivo

```
git status
```

O arquivo constará como "Untracked" (Não monitorado). Mova o arquivo para a Staging Area e execute novamente o validador de estado:

```
git add <nome do arquivo>  
git status
```

Passo 4: Registro Histórico Local (Commit)

Com o arquivo devidamente preparado na Staging Area, realize o commit para salvar a alteração permanentemente no histórico da sua ramificação atual. Escreva uma mensagem concisa e explicativa.

```
git commit -m "Exercício 01"
```

Passo 5: Publicação da Ramificação Remota

Agora, publique a sua branch de trabalho na nuvem. Lembre-se de especificar o nome correto da ramificação criada para não enviar diretamente na main:

```
git push origin <nome_da_branch>
```

Neste momento, entre na interface web do seu GitHub. Você verá que a branch foi publicada com sucesso e está visível na nuvem para os demais membros da equipe.

Passo 6: Unificação do Código (Merge)

Em cenários de desenvolvimento, após a aprovação da funcionalidade, o código contido na branch precisa ser incorporado de volta à linha de produção estável do software (main). Podemos fazer isso na interface do Github. Após a publicação do branch, o Github notifica que há um novo branch, e indaga se queremos comparar e realizar um pull request.

1. Clique em **compare & pull request**
2. Adicione um título e uma descrição, caso necessário
3. Clique em **Create pull request**
4. Se não existirem conflitos, é possível mesclar os códigos
5. Clique em **Merge pull request**
6. Crie uma mensagem de commit
7. Clique em **Confirm merge**

O arquivo aparecerá novamente na pasta, agora integrado com sucesso na sua linha principal estável local.

Passo 7: Eliminação de branches

Pela interface, clique no contador de branches

Na aba Branches, selecione a branch a ser deletada, e clique no ícone da lixeira
branch excluído

Para excluir um branch local, execute o comando

```
git branch -d <nome_da_branch>
```